

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Ejercicios Guiados

Ejercicios Prácticos

Soluciones de Referencia

Introducción y Repaso

- **Sesión anterior (P1)** dominamos:
 - Manipulación avanzada de listas en Python
 - Fundamentos de NumPy: arrays, atributos, operaciones vectorizadas
 - Broadcasting y funciones estadísticas
 - Slicing e indexación de arrays
- **Objetivo de hoy:** Aplicar y consolidar estos conceptos en problemas más elaborados y colaborativos.

- **Consolidar** el uso de listas y arrays de NumPy en problemas físicos
- **Aplicar** análisis de datos experimentales usando ambas herramientas
- **Desarrollar** habilidades de resolución colaborativa
- **Integrar** múltiples conceptos en aplicaciones prácticas de Física/Astronomía

- Listas Python: `append()`, `extend()`, slicing, comprensiones
- Arrays NumPy: `np.array()`, `np.arange()`, `np.linspace()`
- Operaciones Vectorizadas: elementos por elemento sin bucles
- Broadcasting: operaciones entre arrays de diferentes formas
- Funciones Estadísticas: `np.mean()`, `np.std()`, `np.sum()`

Patrón de trabajo

datos $\rightarrow$ procesamiento vectorizado $\rightarrow$ análisis estadístico $\rightarrow$ resultados

Ejercicios Guiados

Ejercicios Guiados $\ni$ Ejercicio Guiado 1:

Comparación de Rendimiento: Listas vs NumPy



Enunciado

- Crear una lista de 1000 números usando `range(1, 1001)`.
- Calcular el cuadrado de cada elemento usando listas por comprensión.
- Repetir el mismo cálculo usando un array de NumPy.
- Comparar los resultados y discutir las diferencias.

Trabajemos este ejercicio **JUNTOS** paso a paso:

1. Crear datos con ambos métodos
2. Aplicar la operación de elevar al cuadrado
3. Verificar que ambos dan el mismo resultado
4. Analizar ventajas de cada enfoque

Ejercicios Guiados $\ni$ Solución Guiado 1:

Comparación Listas vs NumPy



```
1 import numpy as np
2
3 # Método 1: Listas de Python
4 lista_numeros = list(range(1, 1001))
5 cuadrados_lista = [x**2 for x in lista_numeros]
6 print(f"Primeros 5 cuadrados (lista): {cuadrados_lista[:5]}")
7 print(f"Suma total (lista): {sum(cuadrados_lista)}")
8
9 # Método 2: NumPy
10 array_numeros = np.arange(1, 1001)
11 cuadrados_array = array_numeros**2
12 print(f"Primeros 5 cuadrados (NumPy): {cuadrados_array[:5]}")
13 print(f"Suma total (NumPy): {np.sum(cuadrados_array)}")
14
15 # Verificación
16 son_iguales = cuadrados_lista == cuadrados_array.tolist()
17 print(f"¿Los resultados son iguales? {all(son_iguales)}")
18
19 # Análisis estadístico con NumPy
20 print(f"Media: {np.mean(cuadrados_array)}")
21 print(f"Desviación estándar: {np.std(cuadrados_array)}")
```

Discusión: NumPy es más conciso para operaciones matemáticas y ofrece funciones estadísticas integradas.

Ejercicios Guiados $\ni$ Ejercicio Guiado 2:

Análisis de Datos de Temperatura



Enunciado

- Crear un array con temperaturas medidas cada hora durante 24 horas.
- Usar `np.random.normal(20, 3, 24)` para simular datos realistas.
- Calcular estadísticas básicas: media, mínimo, máximo, desviación estándar.
- Identificar las horas con temperaturas extremas ($> \text{media} + \text{std}$, $< \text{media} - \text{std}$).
- Mostrar un resumen completo del análisis.

Conceptos: Simulación de datos, análisis estadístico, indexación condicional.

Física relevante: Análisis de datos meteorológicos y variabilidad térmica.

Ejercicios Guiados $\ni$ Solución Guiado 2:

Análisis de Datos de Temperatura



```
1 import numpy as np
2
3 # Simular datos de temperatura (24 horas)
4 np.random.seed(42) # Para reproducibilidad
5 temperaturas = np.random.normal(20, 3, 24)
6 horas = np.arange(0, 24)
7
8 # Estadísticas básicas
9 media = np.mean(temperaturas)
10 minimo = np.min(temperaturas)
11 maximo = np.max(temperaturas)
12 std_dev = np.std(temperaturas)
13
14 print(f"=== ANÁLISIS DE TEMPERATURAS (24 HORAS) ===")
15 print(f"Media: {media:.2f}°C")
16 print(f"Mínimo: {minimo:.2f}°C (hora {np.argmin(temperaturas)})")
17 print(f"Máximo: {maximo:.2f}°C (hora {np.argmax(temperaturas)})")
18 print(f"Desviación estándar: {std_dev:.2f}°C")
19
20 # Identificar temperaturas extremas
21 temp_altas = temperaturas > (media + std_dev)
22 temp_bajas = temperaturas < (media - std_dev)
23
24 print(f"\n=== TEMPERATURAS EXTREMAS ===")
25 print(f"Horas con temp. alta: {horas[temp_altas]}")
26 print(f"Valores altos: {temperaturas[temp_altas]:.2f}")
27 print(f"Horas con temp. baja: {horas[temp_bajas]}")
28 print(f"Valores bajos: {temperaturas[temp_bajas]:.2f}")
```

Ejercicios Prácticos

Ejercicios Prácticos $\ni$ Ejercicio 1:

Primeros Pasos con NumPy



Enunciado

- Importar la biblioteca NumPy.
- Crear tres arrays diferentes:
 - Un array de 5 elementos con valores 0 usando `np.zeros`.
 - Un array con los enteros del 1 al 10 usando `np.arange`.
 - Un array con 5 valores espaciados uniformemente entre 0 y 1 usando `np.linspace`.
- Imprimir cada array y su forma (shape).

Conceptos: Creación básica de arrays, funciones de NumPy.

Física relevante: Preparación de datos para análisis.

Sugerencia: Explora la documentación de `np.zeros`, `np.arange` y `np.linspace`.



Enunciado

- Crear un array con los números del 1 al 5.
- Realizar operaciones matemáticas elementales:
 - Sumar 10 a todos los elementos.
 - Multiplicar todos los elementos por 2.
 - Elevar todos los elementos al cuadrado.
- Calcular la suma, media, valor máximo y mínimo del array final.

Conceptos: Operaciones vectorizadas básicas, funciones estadísticas simples.

Física relevante: Transformación de datos y estadísticas descriptivas.

Ejercicios Prácticos $\ni$ Ejercicio 3:

Comparación Lista vs NumPy



Enunciado

- Crear una lista Python con números del 1 al 5: `lista = [1, 2, 3, 4, 5]`
- Crear un array NumPy con los mismos valores: `array = np.array([1, 2, 3, 4, 5])`
- Comparar las dos aproximaciones para:
 - Multiplicar todos los elementos por 3
 - Calcular la suma de todos los elementos
- Discutir las diferencias en sintaxis y enfoque

Conceptos: Diferencias entre listas y arrays, vectorización básica.

Física relevante: Eficiencia en procesamiento de datos experimentales.

Sugerencia: Con lista Python necesitarás un bucle o comprensión de lista.

Ejercicios Prácticos $\ni$ Ejercicio 4:

Indexación y Slicing con NumPy



Enunciado

- Crear un array 2D de 4x4 con valores entre 0 y 15 usando `np.reshape`:
 - Partir de `np.arange(16)`
 - Transformarlo en una matriz de 4x4
- Realizar las siguientes operaciones de indexación:
 - Extraer la segunda fila completa
 - Extraer la tercera columna completa
 - Extraer un subarray 2x2 de la esquina inferior derecha
 - Extraer todos los elementos mayores que 8

Conceptos: Arrays multidimensionales, reshape, indexación avanzada.

Física relevante: Manipulación de matrices en análisis de datos físicos.

Sugerencia: La indexación booleana permite filtrar datos según

Ejercicios Prácticos $\ni$ Ejercicio 5:

Análisis de Caída Libre con NumPy



Enunciado

- Simular el movimiento de caída libre de un objeto durante 5 segundos:
 - Crear un array de tiempo con 50 puntos entre 0 y 5 segundos
 - Calcular la posición usando la fórmula $y = y_0 + v_0 t - \frac{1}{2}gt^2$
 - Calcular la velocidad usando $v = v_0 - gt$
- Usar $y_0 = 100$ m, $v_0 = 0$ m/s, $g = 9.8$ m/s²
- Analizar:
 - ¿En qué momento el objeto llega al suelo ($y = 0$)?
 - ¿Cuál es su velocidad máxima?
 - ¿Cuál es la posición a los 3 segundos?

Conceptos: Vectorización, aplicación de fórmulas físicas, análisis de datos.

Física relevante: Mecánica clásica, movimiento bajo gravedad constante.

Soluciones de Referencia

Soluciones de Referencia $\ni$ Solución 1 de Referencia:

Primeros Pasos con NumPy



```
1 import numpy as np
2
3 # Creando diferentes tipos de arrays
4 array_ceros = np.zeros(5)
5 array_enteros = np.arange(1, 11)
6 array_espaciado = np.linspace(0, 1, 5)
7
8 # Imprimiendo arrays y sus formas
9 print("Array de ceros:", array_ceros)
10 print("Forma:", array_ceros.shape)
11 print("\nArray de enteros:", array_enteros)
12 print("Forma:", array_enteros.shape)
13 print("\nArray espaciado:", array_espaciado)
14 print("Forma:", array_espaciado.shape)
15
16 # Resultado esperado:
17 # Array de ceros: [0. 0. 0. 0. 0.]
18 # Forma: (5,)
19 # Array de enteros: [ 1  2  3  4  5  6  7  8  9 10]
20 # Forma: (10,)
21 # Array espaciado: [0.   0.25 0.5  0.75 1. ]
22 # Forma: (5,)
```

Discusión: NumPy ofrece diferentes funciones para crear arrays según nuestras necesidades específicas.

Soluciones de Referencia $\ni$ Solución 2 de Referencia:

Operaciones Básicas con NumPy



```
1 import numpy as np
2
3 # Crear un array con números del 1 al 5
4 arr = np.array([1, 2, 3, 4, 5])
5 print("Array original:", arr)
6
7 # Operaciones elementales
8 arr_suma = arr + 10
9 print("Después de sumar 10:", arr_suma) # [11 12 13 14 15]
10
11 arr_mult = arr_suma * 2
12 print("Después de multiplicar por 2:", arr_mult) # [22 24 26 28 30]
13
14 arr_cuad = arr_mult ** 2
15 print("Después de elevar al cuadrado:", arr_cuad) # [484 576 676 784 900]
16
17 # Estadísticas básicas
18 print("\nEstadísticas del array final:")
19 print("Suma:", np.sum(arr_cuad)) # 3420
20 print("Media:", np.mean(arr_cuad)) # 684.0
21 print("Valor máximo:", np.max(arr_cuad)) # 900
22 print("Valor mínimo:", np.min(arr_cuad)) # 484
```

Discusión: Las operaciones con arrays de NumPy se aplican a todos los elementos sin necesidad de bucles.

Soluciones de Referencia $\ni$ Solución 3 de Referencia:

Comparación Lista vs NumPy



```
1 import numpy as np
2
3 # Crear una lista Python y un array NumPy
4 lista = [1, 2, 3, 4, 5]
5 array = np.array([1, 2, 3, 4, 5])
6
7 # Multiplicar por 3 - Enfoque con listas
8 lista_multiplicada = []
9 for elemento in lista:
10     lista_multiplicada.append(elemento * 3)
11 print("Lista multiplicada:", lista_multiplicada) # [3, 6, 9, 12, 15]
12
13 # Multiplicar por 3 - Enfoque con NumPy
14 array_multiplicado = array * 3
15 print("Array multiplicado:", array_multiplicado) # [3 6 9 12 15]
16
17 # Suma - Enfoque con listas
18 suma_lista = sum(lista)
19 print("Suma de lista:", suma_lista) # 15
20
21 # Suma - Enfoque con NumPy
22 suma_array = np.sum(array)
23 print("Suma de array:", suma_array) # 15
```

Discusión: NumPy permite operaciones vectorizadas más concisas y legibles que las listas de Python.

Soluciones de Referencia $\ni$ Solución 4 de Referencia:

Indexación y Slicing con NumPy



```
1 import numpy as np
2 # Crear un array 2D de 4x4
3 array_2d = np.arange(16).reshape(4, 4)
4 print("Matriz 4x4 original:")
5 print(array_2d)
6 # Resultado:
7 # [[ 0  1  2  3]
8 #  [ 4  5  6  ...]
9 # Extraer la segunda fila completa (índice 1)
10 segunda_fila = array_2d[1, :] # o simplemente array_2d[1]
11 print("\nSegunda fila:")
12 print(segunda_fila) # [4 5 6 7]
13 # Extraer la tercera columna completa (índice 2)
14 tercera_columna = array_2d[:, 2]
15 print("\nTercera columna:")
16 print(tercera_columna) # [ 2  6 10 14]
17 # Extraer subarray 2x2 de la esquina inferior derecha
18 subarray = array_2d[2:4, 2:4]
19 print("\nSubarray 2x2 (esquina inferior derecha):")
20 print(subarray)
21 # [[10 11]
22 #  [14 15]]
23 # Extraer elementos mayores que 8
24 elementos_mayores = array_2d[array_2d > 8]
25 print("\nElementos mayores que 8:")
26 print(elementos_mayores) # [ 9 10 11 12 13 14 15]
```

Discusión: La indexación flexible de NumPy permite extraer datos

Soluciones de Referencia $\ni$ Solución 5 de Referencia:

Análisis de Caída Libre con NumPy



```
1 import numpy as np
2 # Constantes físicas
3 y0 = 100 # altura inicial (m)
4 v0 = 0   # velocidad inicial (m/s)
5 g = 9.8  # aceleración gravitacional (m/s^2)
6 # Crear array de tiempos (0 a 5 segundos, 50 puntos)
7 tiempos = np.linspace(0, 5, 50)
8 # Calcular posición:  $y = y_0 + v_0 t - 0.5 g t^2$ 
9 posiciones = y0 + v0*tiempos - 0.5*g*tiempos**2
10 # Calcular velocidad:  $v = v_0 - g t$ 
11 velocidades = v0 - g*tiempos
12 # Encontrar cuándo el objeto llega al suelo ( $y = 0$ )
13 indices_suelo = np.where(posiciones <= 0)[0]
14 if len(indices_suelo) > 0:
15     tiempo_impacto = tiempos[indices_suelo[0]]
16     velocidad_impacto = velocidades[indices_suelo[0]]
17     print(f"Tiempo de impacto: {tiempo_impacto:.2f} s")
18     print(f"Velocidad al impactar: {velocidad_impacto:.2f} m/s")
19 else:
20     print("El objeto no llega al suelo en el intervalo analizado")
21 # Velocidad máxima (en magnitud)
22 velocidad_max = np.max(np.abs(velocidades))
23 tiempo_vel_max = tiempos[np.argmax(np.abs(velocidades))]
24 print(f"Velocidad máxima: {velocidad_max:.2f} m/s a los {tiempo_vel_max:.2f} s")
25 # Posición a los 3 segundos (aproximadamente)
26 indice_3s = np.argmin(np.abs(tiempos - 3))
27 posicion_3s = posiciones[indice_3s]
28 print(f"Posición a los {tiempos[indice_3s]:.2f} s: {posicion_3s:.2f} m")
```

- Documentación oficial de NumPy
- Matplotlib Docs (tutorial de pyplot)
- Scipy Lectures (capítulo de NumPy y Matplotlib)
- Foros y comunidad: Stack Overflow, Reddit [/r/python](#).

Gracias y hasta la próxima sesión

- Sigam experimentando con NumPy y Matplotlib.
- Cualquier duda, pregunten en foros o a compañeros.
- ¡Nos vemos en la **Semana 7** para avanzar en visualización y análisis!